

UDC: 004.896

## Иерархическая когнитивная архитектура управления для мобильной робототехнической платформы

### Аннотация

Предложена реализация трёхуровневой иерархической когнитивной архитектуры управления мобильным агентом с дифференциальным приводом в среде с препятствиями и переменными свойствами поверхности. Архитектура объединяет нижний уровень управления моторами на основе обучения с подкреплением, средний уровень коррекции на основе линейно-квадратичного регулятора и верхний уровень обхода препятствий по карте. Покомпонентный анализ на двух семействах сценариев показывает, что верхний уровень разгружает нижний при известной геометрии препятствий, но не компенсирует физических ограничений нижнего уровня при слабом сцеплении колёс. Все результаты полностью воспроизводимы.

**Ключевые слова:** обучение с подкреплением, TD3, LQR, иерархическое управление, мобильная робототехника, STRL, residual policy, когнитивный агент

### 1. Введение

Автономная навигация мобильного агента в среде с препятствиями и переменными свойствами поверхности объединяет несколько направлений исследований: обучение с подкреплением, классическое управление, иерархическое планирование. Каждое действует на своём уровне абстракции, и вопрос об их интеграции в единую архитектуру остаётся открытым.

Один из устоявшихся подходов — трёхуровневая иерархическая когнитивная архитектура [1, 2, 3] с распределением поведения по трём уровням, различающимся горизонтом планирования. В диссертации [2] такая архитектура реализована для платформы МП-РМ «Зарница» с  $Q$ -обучением на нижнем уровне; в [4] направление развивается за счёт обработки мультимодальных сигналов.

В настоящей работе предложена реализация трёхуровневой архитектуры для случая *непрерывного* управления. Нижний уровень — нейросетевая стратегия, обученная методом TD3 [5]; средний — LQR-регулятор по линеаризованной модели поперечного отклонения и отклонения курса от прямой к цели; верхний —

модуль обхода препятствий по известной карте. Архитектура протестирована в PyBullet на платформе Husky.

**Вклад работы.** (1) Реализация трёхуровневой иерархической архитектуры для непрерывного управления агентом. (2) Покомпонентный анализ распределения функций между уровнями на 65 конфигурациях  $(7 \times 5 + 6 \times 5)$  с  $n = 10$  запусками на ячейку, показывающий, что верхний уровень разгружает нижний при известной геометрии препятствий, но не компенсирует физических ограничений нижнего уровня. (3) Открытый репозиторий с воспроизводимой реализацией.

## 2. Обзор работ

В задачах непрерывного управления применяются методы актора-критика [6, 5, 7, 8]; в настоящей работе используется TD3 [5] с эталонной реализацией в `stable-baselines3` [9]. Комбинирование обученной стратегии с классическим регулятором [10, 11] компенсирует ограничения каждого подхода в отдельности. Параллельное направление гибридизации обучения с подкреплением и методов оптимального управления систематизировано в обзорах [12, 13], с близкими подходами в [14, 15, 16]. Настоящая работа развивает направление STRL [1, 2, 3, 4] в части непрерывного управления и количественного анализа роли каждого уровня архитектуры.

## 3. Метод

**3.1. Постановка задачи.** Задача формализуется как марковский процесс принятия решений  $\langle S, A, T, R, \gamma \rangle$  с непрерывными пространствами в плоской рабочей области  $10 \times 10$  м с препятствиями известной геометрии. Среда моделируется в PyBullet [17] на платформе Husky. Состояние агента

$$s_t = (\mathbf{r}_t, \theta_t, \mathbf{v}_t, \omega_t, \mathbf{g}_t, \ell_t)^\top \in \mathbb{R}^{n_s}, \quad (1)$$

включает координаты  $\mathbf{r}_t = (x_t, y_t)$ , ориентацию  $\theta_t$ , линейную и угловую скорости  $\mathbf{v}_t, \omega_t$ , относительное положение цели  $\mathbf{g}_t = \mathbf{r}_g - \mathbf{r}_t$ , и лидарные измерения  $\ell_t \in \mathbb{R}^{16}$  (только при наличии препятствий). Действие  $a_t \in [-1, 1]^2$  — нормированные скорости левой и правой пар колёс, преобразуемые в физические  $\omega_w \in [-10, 10]$  рад/с. Шаги физики и управления  $1/240$  и  $1/20$  с,  $\gamma = 0.99$ . Цель на расстоянии 2–5 м, радиус достижения 0.4 м. Эпизод завершается при достижении цели, перевороте, выходе за пределы рабочей области, столкновении или таймауте 500 шагов. Вознаграждение плотное (штраф за расстояние, шаговый штраф, терминальные бонус и штрафы).

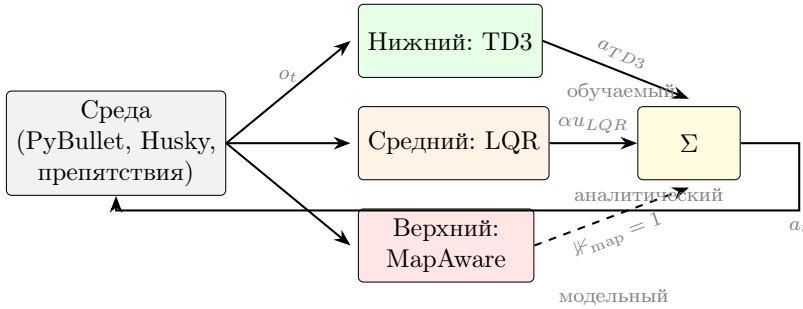


Рис. 1. Трёхуровневая иерархическая архитектура управления. Нижний уровень (TD3) выдаёт нормированное действие  $a_{TD3}$  на каждом шаге. Средний уровень (LQR-residual) выдаёт скалярную коррекцию  $u_{LQR}$  к угловой скорости, комбинируемую с весом  $\alpha$ . Верхний уровень (MapAware) при выполнении условия  $\mathbb{K}_{map}(s_t) = 1$  замещает действие нижнего и среднего уровней согласно формуле (2).

**3.2. Иерархическая архитектура управления.** Архитектура (рис. 1), восходящая к трёхуровневой схеме [1] и развитая для когнитивных агентов в [2, 3], адаптирована для непрерывного управления и включает три уровня.

*Нижний уровень* — обученная методом TD3 стратегия  $\pi_{TD3}(s) \rightarrow a$ , опирающаяся на текущее наблюдение.

*Средний уровень* — LQR-регулятор по линеаризованной модели поперечного отклонения и отклонения курса агента от прямой к цели. Регулятор выдаёт скалярную коррекцию  $u_{LQR} \in [-u_{max}, u_{max}]$  к угловой скорости, комбинируемую с действием нижнего уровня по схеме остаточной коррекции [10, 11].

*Верхний уровень* — модуль обхода препятствий по известной карте (MapAware) с индикатором активации  $\mathbb{K}_{map}(s) \in \{0, 1\}$ . При приближении агента к препятствию в передней полусфере на расстояние ниже  $d_{takeover} = 1.5$  м индикатор переключается в 1, при выходе за  $d_{release} = 2.5$  м — в 0; гистерезис предотвращает дребезг. Пороги согласованы с кинематикой:  $v_{max} = \omega_{max} \cdot r_w \approx 1.8$  м/с при  $\omega_{max} = 10$  рад/с и  $r_w = 0.178$  м, тогда  $d_{takeover}/v_{max} \approx 0.83$  с обеспечивает запас для смены курса. В текущей реализации уровень замещает управление в опасных конфигурациях, не предлагая последовательности подцелей; полноценный планировщик — естественное направление развития.

Конечное действие  $a_t$  на каждом шаге управления формируется правилом

$$a_t = \mathbb{K}_{map}(s_t) \cdot a_{MapAware}(s_t) + (1 - \mathbb{K}_{map}(s_t)) \cdot \text{clip} \left( a_{TD3}(s_t) + \alpha \cdot \begin{bmatrix} -1 \\ +1 \end{bmatrix} u_{LQR}(s_t), -1, 1 \right), \quad (2)$$

где  $\mathbb{K}_{map}$  — индикатор активности верхнего уровня, знаки  $\pm 1$  преобразуют скалярную поправку  $u_{LQR}$  в дифференциальный сигнал на левую и правую пары

колёс,  $\alpha \in [0, 1]$  — фиксированный весовой коэффициент. В настоящей работе используется  $\alpha = 0.2$ . Это значение обеспечивает баланс между подавлением высокочастотных колебаний рулевого управления и сохранением стратегических решений нижнего уровня.

**3.3. Нижний уровень: TD3.** В качестве базовой стратегии используется TD3 [5], актор-критик с двойной оценкой функции ценности, задержанным обновлением актора и буфером переходов. Стратегия  $\pi_\phi$  и функции ценности  $Q_{\theta_1}, Q_{\theta_2}$  — полносвязные сети со слоями 400 и 300 (ReLU), выход стратегии через  $\tanh$ . Обучение функций ценности минимизирует ошибку  $\mathcal{L}(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}}[(Q_{\theta_i}(s, a) - y)^2]$  с целевым значением  $y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_{\phi'}(s') + \varepsilon)$ ,  $\varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma_t), -c, c)$ , где  $\theta'_i, \phi'$  — параметры целевых сетей,  $\sigma_t, c$  — параметры сглаживания. Стратегия обновляется с задержкой  $d$  итераций по детерминированному policy gradient через  $Q_{\theta_1}$ . Гиперпараметры: минибатч 256, Adam с  $\text{lr } 10^{-3}$ ,  $\tau = 0.005$ ,  $d = 2$ ,  $\sigma_t = 0.2$ ,  $c = 0.5$ , шум  $\mathcal{N}(0, 0.3)$ , буфер  $|\mathcal{D}| = 2 \cdot 10^5$ . Использована реализация [9].

Вектор наблюдения  $o_t = (\cos \theta_t, \sin \theta_t, \mathbf{v}_t, \omega_t, \mathbf{g}_t, \ell_t)^\top$  включает ориентацию, скорости и относительное положение цели  $\mathbf{g}_t$ . Лидарные измерения  $\ell_t \in \mathbb{R}^{16}$  получаются агрегированием 360 лучей (паспортные характеристики LDS-01 TurtleBot 3 Burger [18]) в 16 секторов по  $22.5^\circ$  операцией поиска минимума  $\ell_t^{(k)} = \min_{j \in \mathcal{B}_k} d_t^{(j)}$ . Такая агрегация гарантирует обнаружение тонких препятствий, которые могут проваливаться между лучами в низкоразрешённом сканировании. Наблюдение привилегированное, что отделяет управление от восприятия.

**3.4. Средний уровень: LQR-residual.** В системе координат, связанной с прямой от агента к цели, состояние ошибки  $e = [e_y, e_\theta]^\top$  (поперечное отклонение и отклонение курса) при фиксированной опорной скорости  $v_{\text{ref}}$  и малых отклонениях подчиняется линеаризованной модели  $\dot{e} = Ae + B\omega$ , где  $A = \begin{bmatrix} 0 & v_{\text{ref}} \\ 0 & 0 \end{bmatrix}$ ,  $B = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$ . LQR находит закон управления  $\omega = -Ke$ , минимизирующий квадратичный функционал  $J = \int_0^\infty (e^\top Qe + R\omega^2) dt$  с  $Q = \text{diag}(2.0, 1.0)$ ,  $R = 5.0$ . Преобладание штрафа на управление ( $R > Q_{ii}$ ) обеспечивает мягкие коррекции. Матрица  $K$  вычисляется однократно через решение уравнения Риккати.

В реализации  $e_y$  обнуляется на каждом шаге: опорная прямая переопределяется от текущей позы агента к цели, и закон вырождается в  $\omega = -k_\theta e_\theta$ , Р-регулятор по углу с коэффициентом из LQR-задачи. Выход ограничивается  $|\omega| \leq u_{\text{max}} = 0.3$ .

**3.5. Верхний уровень: обход с использованием карты.** Уровень получает карту препятствий в виде списка  $\{(o_i^x, o_i^y, r_i)\}$  и вычисляет расстояния до краёв препятствий в передней полусфере  $\pm\pi/2$  относительно курса агента.

Активация определяется двупороговой логикой с гистерезисом: индикатор  $\mathcal{K}_{\text{map}}$  переключается в 1 при пересечении  $d_{\text{takeover}}$  и обратно в 0 при пересечении  $d_{\text{release}}$ . При активном уровне управление задаётся прямолинейным движением со скоростью  $v_{\text{sup}} = 0.4$  м/с и угловой скоростью  $\omega_{\text{sup}} = k_{\text{turn}} \cdot \text{sign}(-\theta_{\text{nearest}})$ ,  $k_{\text{turn}} = 1.5$ , где  $\theta_{\text{nearest}}$  — угол на ближайшее препятствие. Простота алгоритма преднамеренна: она позволяет изолировать модельное знание о среде. Для сопоставления модельного и сенсорного знания используется также альтернативный модуль обхода по лидарным дальностям (LidarAvoid).

## 4. Эксперименты

**4.1. Конфигурация и метрики.** Эксперименты проведены на сервере 8 ядер Ryzen 9 9950X / 16 ГБ RAM без GPU, PyBullet 3.2 в безголовом режиме, 6 параллельных сред через SubprocVecEnv, сиды  $\{200, \dots, 209\}$ ,  $n = 10$  запусков на ячейку. Основная метрика — доля успешных эпизодов (success rate). Дополнительно фиксируются длина эпизода, гладкость управления, максимальная линейная скорость и доля времени в верхнеуровневом режиме. Сравниваются пять архитектурных режимов: TD3, TD3+LQR, TD3+LidarAvoid, TD3+MapAvoid и TD3+MapAvoid+LQR.

### 4.2. Эксперимент А: множественные конфигурации препятствий.

Семь детерминированных конфигураций препятствий: одиночное препятствие на прямой, два со смещением, коридор из трёх, барьер с проходом, слалом, диагональное расположение цели, широкое препятствие.

| Сценарий         | TD3          | +LQR         | +Lidar       | +Map         | +Map+LQR     |
|------------------|--------------|--------------|--------------|--------------|--------------|
| single_on_line   | 0/10         | 0/10         | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> |
| two_offset       | 0/10         | 0/10         | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> |
| three_corridor   | 0/10         | 0/10         | 0/10         | <b>10/10</b> | <b>10/10</b> |
| barrier_with_gap | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> | 0/10         | 0/10         |
| slalom           | 0/10         | 0/10         | 0/10         | 0/10         | 0/10         |
| diagonal_goal    | 10/10        | 10/10        | 10/10        | 10/10        | 10/10        |
| wide_obstacle    | 0/10         | 0/10         | 0/10         | <b>10/10</b> | <b>10/10</b> |

Таблица 1. Эксперимент А: доля успешных эпизодов (из 10 запусков на сидах 200–209) для семи конфигураций препятствий и пяти архитектурных режимов.

Сценарий *diagonal\_goal* без препятствий на прямой решается всеми режимами и служит контролем. Сценарий *slalom* не решается ни одним режимом, фиксируя границу применимости архитектуры.

В сценариях *single\_on\_line* и *two\_offset* оба варианта верхнего уровня обеспечивают полный успех, тогда как нижний и средний уровни в одиночку не

справляются: обученная стратегия идёт по прямой к цели, а LQR-коррекция общую стратегию не меняет. В *three\_corridor* полный успех достигает только модельный верхний уровень, что соответствует роли стратегического уровня при известной структуре среды. Обратный паттерн наблюдается в *barrier\_with\_gap*, где модельный уровень обходит барьер целиком, хотя геометрический проход проходим напрямую, и в *wide\_obstacle*, где аппроксимация цилиндром малого радиуса не активирует MapAvoid на нужной дистанции. Эти сценарии указывают, что полезность каждого варианта верхнего уровня зависит от соответствия внутренней модели реальной геометрии среды.

**4.3. Эксперимент В: переменные свойства поверхности.** Реализованы три типа поверхности: *NORMAL* (трение 0.9), *ICE* (0.05) и *SAND* (0.6 с дополнительным сопротивлением качению). Каждый тип тестируется в двух конфигурациях, *empty* и *with\_obstacle*,  $n = 10$ .

| Поверхность | Конфиг. | TD3   | +LQR  | +Lidar       | +Map         | +Map+LQR     |
|-------------|---------|-------|-------|--------------|--------------|--------------|
| NORMAL      | empty   | 10/10 | 10/10 | 10/10        | 10/10        | 10/10        |
| NORMAL      | + obst. | 0/10  | 0/10  | 0/10         | <b>10/10</b> | <b>10/10</b> |
| ICE         | empty   | 10/10 | 10/10 | 10/10        | 10/10        | 10/10        |
| ICE         | + obst. | 0/10  | 0/10  | 0/10         | 0/10         | <b>10/10</b> |
| SAND        | empty   | 10/10 | 10/10 | 10/10        | 10/10        | 10/10        |
| SAND        | + obst. | 0/10  | 0/10  | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> |

Таблица 2. Эксперимент В: доля успешных эпизодов (из 10) для трёх типов поверхности и двух конфигураций. Модель TD3 v3 обучена на детерминированном сценарии с препятствием на NORMAL и используется во всех ячейках без переобучения.

В отсутствие препятствий нижний уровень успешен на всех трёх поверхностях, автоматически адаптируя скорость (NORMAL:  $\sim 0.6$  м/с, ICE:  $\sim 0.3$ , SAND:  $\sim 0.5$ ). При наличии препятствия на NORMAL и SAND задачу решает модельный верхний уровень, что согласуется с экспериментом А. Принципиальный результат — сценарий ICE+obstacle, где полного успеха достигает только полная трёхуровневая конфигурация TD3+MapAvoid+LQR: без среднего уровня низкое трение приводит к проскальзыванию при попытке обхода. Это эмпирически подтверждает взаимодополнение уровней — ни один в отдельности не достаточен при физических ограничениях нижнего уровня.

## 5. Обсуждение

*Средний уровень не меняет стратегических решений нижнего уровня.* Включение LQR-регулятора не меняет долю успеха ни в одной ячейке таблиц 1, 2,

однако подавляет высокочастотные колебания курса, что согласуется с теоретической ролью среднего уровня [2, 3].

*Верхний уровень разгружает нижний при адекватной геометрической модели.* В *single\_on\_line* и *two\_offset* верхний уровень обеспечивает полный успех там, где нижний не справляется. Обратный эффект в *barrier\_with\_gap* и *wide\_obstacle* показывает, что полезность модельного знания зависит от точности представления геометрии.

*Модельное знание не компенсирует физических ограничений.* На ICE+obstacle полный успех достигается только полной трёхуровневой архитектурой: верхний уровень предлагает корректную траекторию, но без LQR низкое трение приводит к проскальзыванию.

*Ограничения.* Использован простейший геометрический алгоритм без оптимизационного планирования, нижний уровень обучен с привилегированным наблюдением, реальная платформа не задействована.

## 6. Заключение

Реализована и эмпирически исследована трёхуровневая иерархическая архитектура управления мобильным агентом с дифференциальным приводом, соотносимая с STRL [2, 3]: нижний уровень — стратегия TD3, средний — LQR в схеме остаточной коррекции, верхний — модуль обхода по известной карте. Покомпонентный анализ на семи конфигурациях препятствий и трёх типах поверхности показывает, что средний уровень улучшает гладкость, верхний разгружает нижний при известной геометрии препятствий, но не компенсирует слабое сцепление колёс. Результаты полностью воспроизводимы.

Направления развития: верхний уровень в полной стратегической формулировке с оптимизационным планировщиком подцелей, интеграция с мультимодальной модельной компонентой [4] и перенос на реальную платформу с ROS, требующий адаптации наблюдения (SLAM) и проверки sim-to-real разрыва.

## ЛИТЕРАТУРА

1. Gat E. On three-layer architectures // Artificial Intelligence and Mobile Robots / D. Kortenkamp, R. Bonasso, R. Murphy (eds.). AAAI Press, 1998. P. 195–210.
2. Киселёв Г. А. Интеллектуальная система планирования поведения коалиции робототехнических агентов со STRL архитектурой: дис. . . . канд. техн. наук. Москва, 2020. 117 с.
3. Киселёв Г. А., Панов А. И. Иерархическое психологически-инспирированное планирование для интеллектуальных динамических систем // Известия РАН. Теория и системы управления. 2020. № 1. С. 123–137.
4. Вейценфельд Д. А., Киселёв Г. А. Метод синтеза поведения когнитивного агента на основе обработки мультимодальных сигналов // Моделирование и анализ данных. 2024. Т. 14. № 4. С. 45–62.

5. Fujimoto S., van Hoof H., Meger D. Addressing function approximation error in actor-critic methods // Proc. of ICML. 2018. P. 1587–1596.
6. Lillicrap T., Hunt J., Pritzel A. et al. Continuous control with deep reinforcement learning // Proc. of ICLR. 2016.
7. Haarnoja T., Zhou A., Abbeel P., Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor // Proc. of ICML. 2018. P. 1861–1870.
8. Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal policy optimization algorithms // arXiv preprint arXiv:1707.06347. 2017.
9. Raffin A., Hill A., Gleave A. et al. Stable-Baselines3: Reliable reinforcement learning implementations // Journal of Machine Learning Research. 2021. V. 22. № 268. P. 1–8.
10. Silver T., Allen K., Tenenbaum J., Kaelbling L. Residual policy learning // arXiv preprint arXiv:1812.06298. 2018.
11. Johannink T., Bahl S., Nair A. et al. Residual reinforcement learning for robot control // Proc. of ICRA. 2019. P. 6023–6029.
12. Hewing L., Wabersich K. P., Menner M., Zeilinger M. N. Learning-Based Model Predictive Control: Toward Safe Learning in Control // Annual Review of Control, Robotics, and Autonomous Systems. 2020. V. 3. P. 269–296.
13. Brunke L., Greeff M., Hall A. W., Yuan Z., Zhou S., Panerati J., Schoellig A. P. Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning // Annual Review of Control, Robotics, and Autonomous Systems. 2022. V. 5. P. 411–444.
14. Levine S., Finn C., Darrell T., Abbeel P. End-to-end training of deep visuomotor policies // Journal of Machine Learning Research. 2016. V. 17. P. 1–40.
15. Cheng R., Orosz G., Murray R. M., Burdick J. W. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks // Proc. of AAAI. 2019. P. 3387–3395.
16. Kahn G., Villafior A., Ding B., Abbeel P., Levine S. Self-supervised deep reinforcement learning with generalized computation graphs for robot navigation // Proc. of IEEE ICRA. 2018. P. 5129–5136.
17. Coumans E., Bai Y. PyBullet, a Python module for physics simulation for games, robotics and machine learning. 2016–2021. URL: <http://pybullet.org>
18. Robotis Co., Ltd. LDS-01 (360 Laser Distance Sensor): Spec sheet // URL: [https://manual.robotis.com/docs/en/platform/turtlebot3/appendix\\_lds\\_01/](https://manual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_01/). 2017.